

```
"""Clippeur - service de traitement vidéo.
```

```
POST /process      -> lance un job, renvoie  
{"job_id"}
```

```
GET  /jobs/{id}    -> statut, progression  
et clips générés
```

```
GET  /files/...     -> fichiers MP4 / JPG  
générés
```

```
Variables d'environnement :
```

```
    GROQ_API_KEY      (obligatoire) clé Groq  
    API_KEY           (obligatoire) secret  
partagé avec Lovable, envoyé dans l'en-tête  
X-API-Key
```

```
    PUBLIC_BASE_URL    URL publique du  
service en https (ex.
```

```
https://clippeur.onrender.com)
```

```
    GROQ_LLM_MODEL     modèle de chat Groq  
(défaut : openai/gpt-oss-120b)
```

```
    GROQ_STT_MODEL     modèle de  
transcription (défaut : whisper-large-v3-  
turbo)
```

```
    MAX_VIDEO_MINUTES  durée max de la vidéo  
source (défaut : 240)
```

```
    COOKIES_FILE       chemin d'un fichier  
cookies.txt YouTube (optionnel)
```

```
"""
```

```
import json
```

```
import os
```

```
import re
```

```
import secrets
import shutil
import subprocess
import threading
import time
import uuid
from pathlib import Path

import httpx
import yt_dlp
from fastapi import BackgroundTasks,
Depends, FastAPI, Header, HTTPException,
Request
from fastapi.middleware.cors import
CORSMiddleware
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel

GROQ_KEY = os.environ.get("GROQ_API_KEY",
    "")
API_KEY = os.environ.get("API_KEY", "")
LLM_MODEL =
os.environ.get("GROQ_LLM_MODEL",
    "openai/gpt-oss-120b")
STT_MODEL =
os.environ.get("GROQ_STT_MODEL", "whisper-
large-v3-turbo")
MAX_MIN =
int(os.environ.get("MAX_VIDEO_MINUTES",
    "240"))
COOKIES = os.environ.get("COOKIES_FILE",
    "")
BASE = os.environ.get("PUBLIC_BASE_URL",
    "").rstrip("/")
```

```

DATA = Path(os.environ.get("DATA_DIR",
"/tmp/clippeur")).resolve()
DATA.mkdir(parents=True, exist_ok=True)
GROQ = "https://api.groq.com/openai/v1"
CHUNK = 600 # secondes d'audio envoyées à
la fois à Whisper

JOBS: dict = {}
LOCK = threading.Semaphore(1) # un seul
montage à la fois (CPU limité)

app = FastAPI(title="Clippeur")
app.add_middleware(CORSMiddleware,
allow_origins=["*"], allow_methods=["GET"],
allow_headers=["*"])
app.mount("/files",
StaticFiles(directory=str(DATA)),
name="files")

def auth(x_api_key: str =
Header(default="")):
    if not API_KEY or not
secrets.compare_digest(x_api_key, API_KEY):
        raise HTTPException(401, "Clé API
invalide")

class Req(BaseModel):
    url: str | None = None # lien
YouTube ou Twitch
    file_url: str | None = None # ou lien
direct vers un fichier vidéo importé
    language: str | None = None # ex.

```

```

"fr" (sinon détection automatique)
    max_clips: int | None = None # sinon
calculé selon la durée
    layout: str = "blur"          # "blur"
(fond flouté) ou "fill" (plein cadre)

@app.get("/health")
def health():
    return {"ok": True}

@app.post("/process", dependencies=
[Depends(auth)])
def process(req: Req, bg: BackgroundTasks,
request: Request):
    if not (req.url or req.file_url):
        raise HTTPException(400, "url ou
file_url requis")
    if not GROQ_KEY:
        raise HTTPException(500,
"GROQ_API_KEY manquante sur le serveur")
    cleanup_old()
    jid = uuid.uuid4().hex
    JOBS[jid] = {"id": jid, "status":
"queued", "step": "En attente", "progress":
0,
                    "clips": [], "error":
None}
    base = BASE or
str(request.base_url).rstrip("/")
    bg.add_task(run_job, jid, req, base)
    return {"job_id": jid}

```

```

@app.get("/jobs/{jid}", dependencies=
[Depends(auth)])
def job(jid: str):
    if jid not in JOBS:
        raise HTTPException(404, "Job
introuvable")
    return JOBS[jid]

def cleanup_old():
    for p in DATA.iterdir():
        if p.is_dir() and time.time() -
p.stat().st_mtime > 86400:
            shutil.rmtree(p,
ignore_errors=True)
            JOBS.pop(p.name, None)

def setp(jid, step, progress):
    JOBS[jid].update(step=step,
progress=progress)

def run(cmd, cwd=None):
    r = subprocess.run(cmd,
capture_output=True, text=True, cwd=cwd)
    if r.returncode != 0:
        raise RuntimeError(f"{cmd[0]} a
échoué : {r.stderr[-300:]}")
    return r.stdout

def run_job(jid, req, base):

```

```

d = DATA / jid
d.mkdir(parents=True, exist_ok=True)
with LOCK:
    try:
        JOBS[jid]["status"] = "running"
        setp(jid, "Récupération de la
vidéo en HD", 5)
        src = fetch_video(req, d)
        dur = float(run(["ffprobe", "-
v", "error", "-show_entries",
"format=duration",
                                "-of",
"csv=p=0", str(src)]).strip())
        if dur > MAX_MIN * 60:
            raise ValueError(f"Vidéo
trop longue (maximum {MAX_MIN} minutes)")
        setp(jid, "Transcription", 20)
        words, segs = transcribe(src,
d, req.language)
        if not segs:
            raise ValueError("Aucune
parole détectée dans la vidéo")
        setp(jid, "Analyse des
meilleurs moments", 50)
        moments = pick_moments(segs,
dur, req.max_clips)
        if not moments:
            raise ValueError("Aucun
moment fort trouvé")
        clips = []
        for i, m in enumerate(moments,
1):
            setp(jid, f"Montage du clip
{i}/{len(moments)}", 60 + int(35 * (i - 1)

```

```

/ len(moments)))

clips.append(render_clip(src, d, i, m,
words, req.layout, base, jid))
        JOBS[jid].update(status="done",
step="Terminé", progress=100, clips=clips)
        except Exception as e: # noqa:
BLE001

JOBS[jid].update(status="error",
step="Erreur", error=str(e)[:500])
        finally:
            for pattern in ("source.*",
"audio_*", "*.ass"):
                for f in d.glob(pattern):

f.unlink(missing_ok=True)

# ----- 1. Récupération -----
def fetch_video(req, d):
    if req.file_url:
        out = d / "source.mp4"
        with httpx.stream("GET",
req.file_url, follow_redirects=True,
timeout=600) as r:
            r.raise_for_status()
            with open(out, "wb") as f:
                for chunk in r.iter_bytes(1
<< 20):
                    f.write(chunk)
            return out
    opts = {"format": "bv*
[height<=1080]+ba/b[height<=1080]/b",

```

```

"merge_output_format": "mp4",
    "outtmpl": str(d / "source.%
(ext)s"), "noplaylist": True,
    "quiet": True, "no_warnings":
True}
    if COOKIES and os.path.exists(COOKIES):
        opts["cookiefile"] = COOKIES
    try:
        with yt_dlp.YoutubeDL(opts) as y:
            info = y.extract_info(req.url,
download=False)
            if (info.get("duration") or 0)
> MAX_MIN * 60:
                raise ValueError(f"Vidéo
trop longue (maximum {MAX_MIN} minutes)")
            y.download([req.url])
        except yt_dlp.utils.DownloadError:
            raise RuntimeError("Téléchargement
bloqué par la plateforme. Importe plutôt le
fichier vidéo.")
        files = sorted(d.glob("source.*"))
        if not files:
            raise RuntimeError("Téléchargement
impossible. Importe plutôt le fichier
vidéo.")
        return files[0]

# ----- Appels Groq -----
def groq_post(path, **kw):
    for attempt in range(6):
        r = httpx.post(GROQ + path,
headers={"Authorization": f"Bearer
{GROQ_KEY}"}, timeout=300, **kw)

```

```

        if r.status_code == 429 or
r.status_code >= 500:
            try:
                wait =
float(r.headers.get("retry-after", ""))
            except ValueError:
                wait = 3 * 2 ** attempt
            time.sleep(min(wait, 60))
            continue
        r.raise_for_status()
        return r.json()
    raise RuntimeError("Groq est saturé,
réessaie dans quelques minutes")

# ----- 2. Transcription -----
def transcribe(src, d, lang):
    run(["ffmpeg", "-y", "-i", str(src), "-
vn", "-ac", "1", "-ar", "16000", "-b:a",
"32k",
        "-f", "segment", "-segment_time",
str(CHUNK), "-reset_timestamps", "1",
        str(d / "audio_%03d.mp3")])
    words, segs = [], []
    for n, f in
enumerate(sorted(d.glob("audio_*.mp3"))):
        off = n * CHUNK
        data = [("model", STT_MODEL),
("response_format", "verbose_json"),

("timestamp_granularities[]", "word"),
("timestamp_granularities[]", "segment")]
        if lang:
            data.append(("language", lang))

```

```

        r =
groq_post("/audio/transcriptions",
data=data,

            files={"file":
(f.name, f.read_bytes(), "audio/mpeg"))
        for w in r.get("words", []):
            words.append({"w":
w["word"].strip(), "s": w["start"] + off,
"e": w["end"] + off})
            for s in r.get("segments", []):
                segs.append({"s": s["start"] +
off, "e": s["end"] + off, "t":
s["text"].strip()})
        return words, segs

# ----- 3. Meilleurs moments -----
-
def parse_moments(text):
    m = re.search(r"\{.*\}", text, re.S)
    if not m:
        return []
    try:
        items =
json.loads(m.group(0)).get("moments", [])
    except ValueError:
        return []
    out = []
    for it in items:
        try:
            a, b = float(it["start"]),
float(it["end"])
            if 10 <= b - a <= 120:
                out.append({"start": a,

```

```

"end": b, "score": int(it.get("score",
50)),
                                "title":
str(it.get("title", "Clip"))[:80],
                                "hook":
str(it.get("hook", ""))[:90]})
        except (KeyError, TypeError,
ValueError):
            continue
    return out

def pick_moments(segs, dur, max_clips):
    n = max_clips or max(3, min(12,
round(dur / 600)))
    blocks, cur = [], []
    for s in segs: # blocs d'environ 15
minutes de transcription
        if cur and s["e"] - cur[0]["s"] >
900:
            blocks.append(cur)
            cur = []
            cur.append(s)
    if cur:
        blocks.append(cur)
    per = max(2, min(5, round(n /
len(blocks)) + 1))
    found = []
    for b in blocks:
        text = "\n".join(f"[{s['s']:.0f}]
{s['t']}" for s in b)
        prompt = (
            f"Voici la transcription
horodatée (en secondes) d'un extrait de

```

```

vidéo. Choisis jusqu'à {per} "
    "moments forts (punchlines,
    rires, émotion, révélation, tension, grosse
    réaction) qui "
    "fonctionnent seuls. Chaque
    moment dure entre 15 et 90 secondes,
    commence au début d'une "
    "phrase et finit à la fin d'une
    phrase. Pour chacun donne : start et end
    (secondes), "
    "score (0 à 100, potentiel
    viral), title (court), hook (phrase
    d'accroche de 4 à 9 mots, "
    "dans la langue de la
    transcription, qui donne envie de
    regarder). Réponds uniquement avec "
    'ce JSON : {"moments":
    [{"start":0,"end":0,"score":0,"title":"","h
    ook":""}]\n\n' + text)
    r = groq_post("/chat/completions",
    json={
        "model": LLM_MODEL,
        "temperature": 0.3,
        "messages": [{"role": "system",
        "content": "Tu es un monteur expert de
        clips viraux TikTok. "

        "Tu réponds uniquement en JSON
        valide."},
        {"role": "user",
        "content": prompt}}])
    found += parse_moments(r["choices"]
    [0]["message"]["content"])
    starts = [s["s"] for s in segs]

```

```

        ends = [s["e"] for s in segs]
        for m in found: # aligne sur les
limites de phrases
            m["start"] = max(0, min(starts,
key=lambda x: abs(x - m["start"])))
            m["end"] = min(dur, min(ends,
key=lambda x: abs(x - m["end"])))
            found = [m for m in found if m["end"] -
m["start"] >= 8]
            chosen = []
            for m in sorted(found, key=lambda x: -
x["score"]): # garde les meilleurs sans
chevauchement
                if all(m["start"] >= c["end"] or
m["end"] <= c["start"] for c in chosen):
                    chosen.append(m)
                if len(chosen) >= n:
                    break
            return sorted(chosen, key=lambda x:
x["start"])

# ----- 4. Montage -----
def ass_time(t):
    t = max(t, 0)
    return f"{int(t // 3600)}:{int(t % 3600
// 60):02d}:{t % 60:05.2f}"

def esc(t):
    return t.replace("\\", "").replace("{",
"(").replace("}", ")").replace("\n", " ")

```

```

def build_ass(path, words, hook, a, b):
    length = b - a
    out = [
        "[Script Info]", "ScriptType:
v4.00+", "PlayResX: 1080", "PlayResY:
1920", "WrapStyle: 0", "",
        "[V4+ Styles]",
        "Format: Name, Fontname, Fontsize,
PrimaryColour, SecondaryColour,
OutlineColour, BackColour, "
        "Bold, Italic, Underline,
StrikeOut, ScaleX, ScaleY, Spacing, Angle,
BorderStyle, Outline, "
        "Shadow, Alignment, MarginL,
MarginR, MarginV, Encoding",
        "Style: Cap,DejaVu
Sans,78,&H00FFFFFF,&H00FFFFFF,&H00000000,&H
64000000,1,0,0,0,100,100,0,0,1,7,2,2,60,60,
420,1",
        "Style: Hook,DejaVu
Sans,68,&H00FFFFFF,&H00FFFFFF,&H00FF5C7B,&H
00FF5C7B,1,0,0,0,100,100,0,0,3,18,0,8,80,80
,260,1",
        "", "[Events]",
        "Format: Layer, Start, End, Style,
Name, MarginL, MarginR, MarginV, Effect,
Text",
    ]
    if hook: # hook animé pendant les 3
premières secondes
        out.append(f"Dialogue: 1,
{ass_time(0)},{ass_time(min(3.2,
length))},Hook,,0,0,0,, "
                    f"

```

```

{{\\fad(150,300)\\fscx70\\fscy70\\t(0,250,\\
\\fscx100\\fscy100)}}{esc(hook)}")
    out.append(f"Dialogue: 2,{ass_time(0)},
{ass_time(length)},Cap,,0,0,0,, " # barre
de progression
        f"
{{\\an7\\pos(0,1904)\\p1\\c&H00FF5C7B&\\fsc
x0\\t(0,{int(length * 1000)},\\fscx100)}}"
        "m 0 0 1 1080 0 1 1080 16 1
0 16{\\p0}")
    ws = [w for w in words if w["e"] > a
and w["s"] < b]
    for g in range(0, len(ws), 3): # sous-
titres mot par mot, 3 mots à l'écran
        grp = ws[g:g + 3]
        for i, w in enumerate(grp):
            s = max(w["s"] - a, 0)
            if i + 1 < len(grp):
                e = grp[i + 1]["s"] - a
            else: # dernier mot du groupe
: s'arrête avant le groupe suivant
                nxt = ws[g + 3]["s"] - a if
g + 3 < len(ws) else length
                e = min(w["e"] - a + 0.15,
nxt, length)
            if e <= s:
                e = s + 0.1
            txt = " ".join((f"
{{\\c&H0000FFFF&}}{esc(x['w'])}
{{\\c&H00FFFFFF&}}" if j == i else
esc(x["w"])))
                                for j, x in
enumerate(grp))
        out.append(f"Dialogue: 0,

```

```

{ass_time(s)},{ass_time(e)},Cap,,0,0,0,,
{txt}")
    path.write_text("\n".join(out),
encoding="utf-8")

def render_clip(src, d, i, m, words,
layout, base, jid):
    a, b = m["start"], m["end"]
    length = b - a
    ass = d / f"subs_{i}.ass"
    build_ass(ass, words, m["hook"], a, b)
    if layout == "fill":
        graph = "
[0:v]scale=1080:1920:force_original_aspect_
ratio=increase,crop=1080:1920,setsar=1[v0]"
    else:
        graph = ("[0:v]split=2[x][y];"
                "
[x]scale=1080:1920:force_original_aspect_ra
tio=increase,crop=1080:1920,boxblur=40:6[bg
];"
                "[y]scale=1080:-2[fg];[bg]
[fg]overlay=(W-w)/2:(H-h)/2,setsar=1[v0]")
        graph += f";[v0]ass={ass.name}[v]"
    out = d / f"clip_{i}.mp4"
    run(["ffmpeg", "-y", "-ss", f"{a:.2f}",
        "-t", f"{length:.2f}", "-i", str(src),
        "-filter_complex", graph, "-map",
        "[v]", "-map", "0:a?", "-r", "30",
        "-c:v", "libx264", "-preset",
        "veryfast", "-crf", "20", "-pix_fmt",
        "yuv420p",
        "-c:a", "aac", "-b:a", "160k", "-

```

```
movflags", "+faststart", str(out)],
cwd=str(d))
    thumb = d / f"thumb_{i}.jpg"
    run(["ffmpeg", "-y", "-ss", "1", "-i",
str(out), "-frames:v", "1", "-vf",
"scale=360:-2", str(thumb)])
    return {"index": i, "title":
m["title"], "hook": m["hook"], "score":
m["score"],
        "start": round(a, 1), "end":
round(b, 1), "duration": round(length, 1),
        "url": f"
{base}/files/{jid}/clip_{i}.mp4",
"thumbnail": f"
{base}/files/{jid}/thumb_{i}.jpg"}
```